GitHub Actions & Terraform Workflow

A Clear, Step-by-Step Explanation for Beginners

This reference guide breaks down your production-grade GitHub Actions workflow line-by-line. It uses Infrastructure as Code (IaC) with Terraform to safely manage cloud resources on Amazon Web Services (AWS).

1. Trigger & Safety Guardrails (The Metadata)

Workflow Name & Triggers

```
name: Terraform

on:
  pull_request:
    paths: ["terraform/**", ".github/workflows/terraform.yml"]
  push:
    branches: [main]
    paths: ["terraform/**", ".github/workflows/terraform.yml"]
```

- **Explanation:** The automation robot watches your repository. It only wakes up if a Pull Request is opened or code is pushed directly to the `main` branch, **AND** the changes happen inside the `terraform/` folder or to this workflow file itself. If you only edit a text file outside this directory, the robot stays asleep to save computing resources.

Security Clearances

```
permissions:
  id-token: write
  contents: read
```

- **Explanation:** Allows GitHub to securely connect to AWS using **OIDC (OpenID Connect)**. Instead of using highly dangerous permanent passwords or keys, this generates a safe, temporary security token for AWS. It also gives the robot permission to read your repository code.

Concurrency Traffic Control

```
concurrency:
  group: terraform-${{ github.ref }}
  cancel-in-progress: false
```

- **Explanation:** This stops multiple Terraform tasks from clashing on the same branch. If you push code twice rapidly, the second run will wait patiently for the first run to completely finish. This ensures your cloud setup doesn't get locked up or corrupted mid-way.

Environment Variables

```
env:
  AWS_REGION: ap-south-1
  STACKS: "network database compute cdn serverless-audit serverless-contact
observability"
```

- **Explanation:** Defines the AWS deployment region (ap-south-1 = Mumbai) and lists a specific, dependency-ordered sequence of modules to build. Your network setup builds first, followed by the database, followed by compute servers, ensuring everything stays structured.

2. The Infrastructure Server (The Job)

```
jobs:
  terraform:
    runs-on: ubuntu-latest
```

- **Explanation:** Tells GitHub to provision a brand-new, fresh **Ubuntu Linux** cloud runner server. Every step written below will execute inside this temporary virtual machine.

3. Step-by-Step Execution Sequence

Step 1: Download Code

```
- uses: actions/checkout@v4
```

Action: Copies your codebase directly onto the empty Ubuntu server runner so it can read and execute your files.

Step 2: Log into AWS Securely

```
- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${{ vars.AWS_DEPLOY_ROLE_ARN }}
    aws-region: ${{ env.AWS_REGION }}
```

Action: Logs the runner server securely into your AWS account using your pre-defined deployment role, allowing it to create live resources.

Step 3: Setup Terraform CLI

```
- uses: hashicorp/setup-terraform@v3
  with:
    terraform_version: 1.9.0
```

Action: Installs **Terraform version 1.9.0** directly onto the clean Ubuntu server so it understands how to run your orchestration commands.

Step 4: The Evaluation Loop (Plan vs Apply)

```
- name: Plan or Apply each stack
  run: |
    set -euo pipefail
    for stack in $STACKS; do
      echo "::group::terraform $stack"
      cd "terraform/$stack"
      terraform init -input=false
      if [[ "${{ github.event_name }}" == "pull_request" ]]; then
        terraform plan -input=false -no-color -lock-timeout=5m
      else
        terraform apply -input=false -auto-approve -lock-timeout=5m
      fi
      cd - > /dev/null
      echo "::endgroup::"
    done
```

Action: Runs a custom script that loops through each folder in your `STACKS` list:

- `set -euo pipefail` stops the pipeline instantly if any error happens so it doesn't break your cloud infrastructure.
- `terraform init` downloads provider plugins inside the module folder.
- **The Core Logic:** If the event is a **Pull Request**, it runs `terraform plan` (a safe dry-run simulation to preview changes). If it is a direct push/merge to **main**, it runs `terraform apply` to push the changes live onto AWS.

Summary of Automation Benefits

- **Pull Requests:** Acts as an automated code reviewer, simulating exactly what infrastructure will change before you merge it.
- **Merges to Main:** Acts as your automated deployment engineer, pushing configuration updates securely and sequentially live to AWS.